Security Audit & Performance Optimization Report

GreenLeaf Inventory Management API (MERN / Node.js / Express / MongoDB)

Report type: Security review and remediation implementation guide

Prepared: 18 October 2026

Scope: API authentication and authorization, injection, mass assignment, input validation, analytics caching, query optimization, and repeatable performance measurement.

Important accuracy note: This report provides a complete implementation and verification plan for the described assignment. The actual source code, database, and test-run output were not supplied in this request, so it would be inaccurate to claim that vulnerabilities have already been fixed or that specific performance gains have already been measured. Replace the marked measurement fields with results from your running application before submitting this as evidence of completed testing.

1. Executive Summary

The audit should focus on three security risks explicitly named in the assignment: injection, mass assignment, and broken access control. The API should reject invalid or unexpected input, authorize every protected operation on the server, and return only fields the current role is allowed to access. Analytics endpoints are good candidates for short-lived caching because they repeatedly calculate aggregate values from sales and inventory data.

Completion criteria and evidence

Required outcome	Evidence needed
Security audit with severity ratings	Findings register in Section 3, plus dated test results
Critical/high issues remediated	Code changes, regression tests, and no remaining open critical/high findings
Before/after performance report	Same workload, same environment, multiple runs, recorded p50/p95/average latency
Analytics caching implemented	Cache middleware/service, TTL, cache invalidation, hit/miss evidence
Validation and sanitization on all API endpoints	Shared validation middleware and a route-by-route coverage checklist

Do not report a finding as “fixed” solely because code was added. Retest the original exploit attempt and record the observed HTTP status and response.

2. Scope and Audit Method

In-scope API areas

• Authentication: login, token refresh, logout, and current-user endpoints.

• Users: read, create, update, delete, role changes.

• Plants/inventory: list, detail, create, update, delete, search, category/status filters.

• Sales: list, create, detail, reports, and analytics/aggregate endpoints.

• Categories and other reference data.

• Shared middleware: authentication, role authorization, validation, error handling, rate limiting, and caching.

Test method

1. Inventory every route and its HTTP method.
2. Record whether it requires a valid token and which roles may access it.
3. Test with no token, malformed token, expired token, low-privilege token, and authorized token.
4. Send malicious query/body values and unexpected fields.
5. Capture response status, response body, and server logs.
6. Fix the issue, add a regression test, and repeat the exact test.
7. Benchmark the same endpoints before and after optimization using the same dataset and load profile.

Only test systems and accounts you are authorized to assess.

3. Security Findings Register

Severity ratings below are **risk ratings to validate during testing**, not claims that each vulnerability was observed in the supplied application.

F-01 — Broken access control / missing role enforcement

Suggested severity: Critical if a normal user can perform administrator-only actions; otherwise High where sensitive data or write actions are exposed.

Risk: A user may access another user's records, change roles, delete inventory, or retrieve reports without the required role.

Test:

• Call protected endpoints without an `Authorization` header.

• Repeat with a malformed/expired token.

• Repeat using a valid low-privilege user token against admin-only routes.

• Try changing a resource ID to another user's/resource's ID.

• Try updating one's own role to `ADMIN`.

Expected secure result: `401 Unauthorized` when authentication is absent/invalid; `403 Forbidden` when authenticated but not authorized; `404 Not Found` can be used for resources the caller is not allowed to discover. No unauthorized state change or sensitive data disclosure.

Remediation:

• Authenticate in middleware, then enforce roles on every protected route.

• Never trust a role, owner ID, or permission sent in the request body.

• Apply ownership checks at the resource query level where relevant.

• Deny access by default.

F-02 — Mass assignment / over-posting

Suggested severity: High when privileged fields such as `role`, `isAdmin`, `owner`, `price`, or stock controls can be modified; Medium for non-sensitive unexpected fields.

Risk: Passing `req.body` directly into `Model.create()` or `findByIdAndUpdate()` may allow clients to change fields the endpoint was not designed to accept.

Unsafe pattern:

```
await User.findByIdAndUpdate(req.params.id, req.body, { new: true });
```

Remediation: Explicitly allowlist editable fields. Do not accept `role`, password hash, ownership, or internal flags from ordinary profile-update endpoints. Use `{ runValidators: true, new: true }` with Mongoose updates.

F-03 — NoSQL injection / unsafe query construction

Suggested severity: High if query operators can bypass authentication or filters; Medium otherwise.

Risk: MongoDB operators such as `$ne`, `$gt`, or `$where`, or untrusted objects in filters, can change query meaning if user input is passed directly into a query.

Test payload examples (in a test environment):

```
{"email":{"$ne":null},"password":{"$ne":null}}
```

```
?category[$ne]=anything
```

Also test JSON values where a string is expected, for example an object in place of `plantId` or `status`.

Expected secure result: Invalid input returns `400 Bad Request`; no login bypass, broadened query, or unauthorized data access occurs.

Remediation: Validate types and allowed values; construct MongoDB filters from validated scalar values only; use `express-mongo-sanitize` where compatible with the current stack; do not use `$where` or string-built database queries.

F-04 — Missing or inconsistent input validation

Suggested severity: High if invalid input can affect access control, financial/sales values, or data integrity; Medium for ordinary invalid records.

Risk: Invalid IDs, negative quantities, invalid dates, oversized strings, malformed email addresses, and unknown enum values may create corrupt records or cause errors.

Remediation: Validate params, query strings, and request bodies at every route. Validate ObjectId syntax before database lookup. Apply domain rules such as quantity > 0, price >= 0, rating within the allowed range, and dates in the accepted format.

F-05 — Sensitive error messages or secrets in logs/responses

Suggested severity: Medium, potentially High if secrets or tokens are exposed.

Risk: Stack traces, database details, passwords, refresh tokens, or environment variables may leak to clients or logs.

Remediation: Return a generic production error response; log a request/correlation ID and safe diagnostic details server-side; never log credentials or tokens.

F-06 — Analytics endpoint abuse / excessive query cost

Suggested severity: Medium; can become High if it enables service disruption.

Risk: Expensive aggregation pipelines, unbounded date ranges, and repeated requests can increase database load.

Remediation: Bound date ranges and result sizes, validate filters, add appropriate indexes, cache repeatable analytics responses, and rate-limit expensive endpoints.

4. Implementation Examples (Express + Mongoose)

Adapt filenames and model fields to the actual project. These snippets are reference implementations; integrate them with the existing error-handling conventions rather than creating duplicate middleware.

4.1 Authentication and role authorization

```
// middleware/authMiddleware.js
const jwt = require("jsonwebtoken");
const User = require("../models/User");

async function authenticate(req, res, next) {
  try {
    const header = req.headers.authorization || "";
    const [scheme, token] = header.split(" ");

    if (scheme !== "Bearer" || !token) {
      return res.status(401).json({
        success: false,
        message: "Authentication required"
      });
    }

    const payload = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
    const user = await User.findById(payload.sub).select("_id role isActive");

    if (!user || user.isActive === false) {
      return res.status(401).json({
        success: false,
        message: "Invalid authentication"
      });
    }

    req.user = { id: user._id.toString(), role: user.role };
    next();
  } catch (error) {
    return res.status(401).json({
      success: false,
      message: "Invalid or expired token"
    });
  }
}

const authorize = (...allowedRoles) => (req, res, next) => {
  if (!req.user) {
    return res.status(401).json({ success: false, message: "Authentication required" });
  }

  if (!allowedRoles.includes(req.user.role)) {
    return res.status(403).json({ success: false, message: "Forbidden" });
  }

  next();
};

module.exports = { authenticate, authorize };
```

Example route use:

```
router.get("/users", authenticate, authorize("ADMIN"), getUsers);
router.post("/plants", authenticate, authorize("ADMIN", "MANAGER"), createPlant);
router.get("/sales/analytics", authenticate, authorize("ADMIN", "MANAGER"), getSalesAnalytics);
```

Use the exact role values defined in the application's User schema. If the schema uses `Admin` rather than `ADMIN`, make the code consistent with that schema.

4.2 Allowlist updates to prevent mass assignment

```
// Example: updating a user's profile
const allowedFields = ["name", "email", "phone"];
const updates = {};

for (const field of allowedFields) {
  if (Object.prototype.hasOwnProperty.call(req.body, field)) {
    updates[field] = req.body[field];
  }
}

const user = await User.findByIdAndUpdate(
```

```
  req.params.id,
  { $set: updates },
  { new: true, runValidators: true, context: "query" }
).select("-password -refreshToken");

if (!user) {
  return res.status(404).json({ success: false, message: "User not found" });
}

res.json({ success: true, data: user });
```

Important:

- Use a separate admin-only endpoint for role changes.
- Verify whether the caller may update the target user.
- Do not return password hashes or refresh tokens.
- Do not spread `req.body` into a database update.

4.3 Validation and sanitization

A shared validation library such as `express-validator` or `zod` is recommended. Example using `express-validator`:

```
const { param, query, body, validationResult } = require("express-validator");

function validateRequest(req, res, next) {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      success: false,
      message: "Validation failed",
      errors: errors.array().map(({ path, msg }) => ({ field: path, message: msg }))
    });
  }
  next();
}

const objectIdParam = (name) =>
  param(name).isMongoId().withMessage("Invalid ID");

const paginationRules = [
  query("page").optional().isInt({ min: 1, max: 100000 }).toInt(),
  query("limit").optional().isInt({ min: 1, max: 100 }).toInt(),
  validateRequest
];

const createSaleRules = [
  body("plantId").isMongoId(),
  body("quantity").isInt({ min: 1, max: 100000 }).toInt(),
  body("totalPrice").isFloat({ min: 0 }).toFloat(),
  body("saleDate").optional().isISO8601().toDate(),
  validateRequest
];

module.exports = { validateRequest, objectIdParam, paginationRules, createSaleRules };
```

Apply validators on every route that accepts `req.params`, `req.query`, or `req.body`, not only create routes. Validate enum values against explicit allowlists. Add maximum lengths and reasonable bounds to text and numeric fields. Sanitization is not a substitute for validation or authorization.

Example safe filter construction:

```
const allowedStatuses = ["ACTIVE", "INACTIVE"];
const filter = {};

if (req.query.status !== undefined) {
  if (!allowedStatuses.includes(req.query.status)) {
    return res.status(400).json({ success: false, message: "Invalid status" });
  }
  filter.status = req.query.status;
}
```

For login, validate `email` as a string and password as a string with a sensible maximum length; do not strip or mutate passwords in ways that change legitimate credentials.

4.4 NoSQL injection defenses

```
// Never do this:
const plants = await Plant.find(req.query);

// Prefer validated, explicit fields:
const filter = {};

if (typeof req.query.category === "string" && req.query.category.length <= 80) {
  filter.category = req.query.category;
}
if (typeof req.query.status === "string") {
  if (!["ACTIVE", "INACTIVE"].includes(req.query.status)) {
    return res.status(400).json({ success: false, message: "Invalid status" });
  }
  filter.status = req.query.status;
}

const plants = await Plant.find(filter).limit(100);
```

Install and configure `express-mongo-sanitize` if it is compatible with the Express/Mongoose versions in use. Confirm its current configuration and test behavior before deploying. Also consider `helmet`, request-size limits, and rate limiting. Keep dependencies updated.

4.5 Analytics caching with in-memory cache

For a single Node.js instance, `node-cache` is a simple option. Its cache is local to that process and is cleared on restart; it is not shared across multiple Render instances. Use Redis for multi-instance deployments or when cache persistence/shared invalidation is required.

```
npm install node-cache
```

```
// utils/analyticsCache.js
const NodeCache = require("node-cache");

const analyticsCache = new NodeCache({
  stdTTL: 60,
  checkperiod: 120,
  useClones: false
});

function makeCacheKey(prefix, params) {
  const normalized = Object.keys(params)
    .sort()
    .reduce((result, key) => {
      result[key] = params[key];
      return result;
    }, {});
  return `${prefix}:${JSON.stringify(normalized)}`;
}

module.exports = { analyticsCache, makeCacheKey };
```

Example analytics controller:

```
const { analyticsCache, makeCacheKey } = require("../utils/analyticsCache");

async function getSalesAnalytics(req, res, next) {
  try {
    // Validate and normalize these values before using them.
    const from = req.query.from || "";
    const to = req.query.to || "";
    const cacheKey = makeCacheKey("sales-analytics-v1", { from, to });

    const cached = analyticsCache.get(cacheKey);
    if (cached) {
      res.set("X-Cache", "HIT");
      return res.status(200).json(cached);
    }

    const data = await Sale.aggregate([
      // Add validated date match and project-specific aggregation stages here.
      { $group: {
          _id: null,
          totalSales: { $sum: "$totalPrice" },
          totalUnits: { $sum: "$quantity" },
          saleCount: { $sum: 1 }
      }},
      { $project: { _id: 0, totalSales: 1, totalUnits: 1, saleCount: 1 } }
    ]);
```

```
    const response = { success: true, data: data[0] || {
      totalSales: 0, totalUnits: 0, saleCount: 0
    }};

    analyticsCache.set(cacheKey, response, 60);
    res.set("X-Cache", "MISS");
    return res.status(200).json(response);
  } catch (error) {
    next(error);
  }
}
```

Required integration notes:

• Add the validated `from`/`to` `$match` stage before `$group`; reject invalid dates and excessively wide ranges.

• Include every response-affecting filter in the cache key. If results vary by tenant/user/permissions, include that identity/authorization scope in the key or do not share cached results.

• Invalidate relevant keys after creating/updating/deleting sales, or keep a short TTL and document the accepted staleness.

• Do not cache login, token refresh, user-specific sensitive responses, or errors.

• Use a stable cache key version (`v1`) so future aggregation changes can invalidate old keys.

• Track cache hits/misses and compare database execution time with cache-hit response time.

4.6 Query optimization and indexes

Indexes must match actual query filters and sort order. Confirm with MongoDB `explain("executionStats")`; do not add indexes blindly because indexes consume memory/disk and slow writes.

Example candidates to evaluate against real query patterns:

```
// Plant list filtered by category/status and sorted by creation date:
PlantSchema.index({ category: 1, status: 1, createdAt: -1 });

// Sales reports filtered by saleDate:
SaleSchema.index({ saleDate: -1 });

// Reports filtering by date and plant:
SaleSchema.index({ plantId: 1, saleDate: -1 });
```

Use only the indexes supported by observed query patterns. If the query uses a different sort or filter order, adjust the compound index accordingly. Avoid redundant indexes.

Before and after:

```
const result = await Sale.find({ saleDate: { $gte: from, $lte: to } })
  .select("plantId quantity totalPrice saleDate")
  .sort({ saleDate: -1 })
  .limit(100)
  .explain("executionStats");

console.log({
  executionTimeMillis: result.executionStats.executionTimeMillis,
  totalDocsExamined: result.executionStats.totalDocsExamined,
  totalKeysExamined: result.executionStats.totalKeysExamined,
  nReturned: result.executionStats.nReturned
});
```

Use `lean()` for read-only Mongoose queries where document methods/getters are not required. Select only required fields, use bounded pagination, and avoid N+1 query patterns. For aggregation, put selective `$match` stages early and inspect the winning plan.

5. Unauthorized Access Test Matrix

Run each test with controlled test accounts and record actual results.

ID	Request/test	Expected result	Pass condition
AC-01	Protected endpoint, no token	401	No protected data returned
AC-02	Malformed/expired token	401	Request rejected
AC-03	Employee/client token on admin-only endpoint	403	No state change
AC-04	Low-role user changes own role to ADMIN	400 or 403	Role remains unchanged
AC-05	User reads/updates another user's private record	403 or 404	No disclosure or mutation
AC-06	User deletes plant/sale without permission	403	Record remains
AC-07	Request contains forged `ownerId`/`soldBy`	400 or ignored	Server-derived identity is used
AC-08	Valid authorized admin request	2xx	Expected operation succeeds

For GreenLeaf, align these cases with the actual roles in the backend schema (for example `Admin`, `Manager`, `Employee`, or the exact uppercase values used by the frontend). Role names are case-sensitive unless normalized deliberately.

6. Injection, Validation, and Mass-Assignment Test Matrix

ID	Test input	Expected result
IV-01	Query value supplied as an object/operator, e.g. `status[$ne]=x`	400; query is not broadened
IV-02	JSON field expected to be a string supplied as `{"name":{"$ne":null}}`	400
IV-03	Invalid MongoDB ObjectId in path	400
IV-04	Negative or zero sale quantity	400
IV-05	Unsupported status/category enum	400
IV-06	Oversized text field	400
IV-07	Extra field such as `{"role":"ADMIN"}` in profile update	Rejected or ignored; role unchanged
IV-08	Invalid date or `from` later than `to`	400
IV-09	Excessive page limit or unbounded analytics range	400 or safely bounded
IV-10	HTML/script-like text in a plain-text field	Stored/rendered safely; output encoded in UI

For HTML content, avoid accepting HTML unless the product requires it. If HTML is required, use a maintained HTML sanitizer with an explicit allowlist. For ordinary text fields, validate length/type and rely on context-appropriate output encoding; do not assume stripping characters alone prevents XSS.

7. Performance Measurement Plan

Baseline capture (before changes)

Choose representative endpoints, for example:

- `GET /api/plants?page=1&limit=20`
- `GET /api/sales?page=1&limit=20`

- `GET /api/sales/analytics?from=YYYY-MM-DD&to=YYYY-MM-DD`

Use the same database snapshot, machine/deployment size, request parameters, network location, and concurrency for baseline and post-change tests. Warm up the app first, then run at least 5 repeated test batches. Record average and p95 latency; use p95 as the primary user-facing latency metric. Also record HTTP error rate and MongoDB docs/keys examined.

Results table — fill with real measurements

Endpoint	Before avg (ms)	Before p95 (ms)	After avg (ms)	After p95 (ms)	Cache hit rate / DB stats
Plant list	___	___	___	___	Docs examined: ___
Sales list	___	___	___	___	Docs examined: ___
Sales analytics (cache MISS)	___	___	___	___	Keys/docs examined: ___
Sales analytics (cache HIT)	N/A	N/A	___	___	Hit rate: ___%

Do not invent numbers. If you need a percentage improvement, calculate it from measured values:

Latency reduction (%) = `((before p95 - after p95) / before p95) × 100`

Example only: if p95 changes from 400 ms to 240 ms, reduction = 40%. This is a calculation example, not a result for your application.

Example k6 script

Install k6, set `BASE_URL` and a valid test token, then run a controlled test. Use a non-production test environment.

```
// performance.js
import http from "k6/http";
import { check, sleep } from "k6";

export const options = {
  vus: 10,
  duration: "60s",
  thresholds: {
    http_req_failed: ["rate<0.01"],
    http_req_duration: ["p(95)<1000"]
  }
};

const base = __ENV.BASE_URL;
const token = __ENV.TOKEN;

export default function () {
  const headers = { Authorization: `Bearer ${token}` };
  const endpoints = [
    `${base}/api/plants?page=1&limit=20`,
    `${base}/api/sales?page=1&limit=20`,
    `${base}/api/sales/analytics?from=2026-01-01&to=2026-10-01`
  ];

  for (const url of endpoints) {
    const response = http.get(url, { headers });
    check(response, {
      "status is 200": (r) => r.status === 200
    });
  }
  sleep(1);
}
```

Run the same script before and after changes. For analytics caching, run a separate repeat-request test and inspect `X-Cache: MISS` on the first request and `X-Cache: HIT` on subsequent requests. Do not treat the sample threshold as proof of performance; use the baseline and your target SLA.

8. Endpoint Validation Coverage Checklist

Mark each item only after reviewing the route and adding a test.

- ■ Authentication routes: validate email/password types, lengths, rate limits, and token lifecycle.
- ■ User routes: validate IDs, names, email, role allowlist, and allowed update fields.
- ■ Plant/inventory routes: validate IDs, category/status enums, numeric stock/price bounds, search length, page and limit.
- ■ Sale routes: validate plant ID, quantity, total price, sale date, and server-derived `soldBy`.
- ■ Category routes: validate name, slug, status, and image/reference IDs.
- ■ Analytics routes: validate date format/range, filters, output size, and authorization.
- ■ File upload routes (if present): validate MIME type and size; do not trust filename or client-provided content type alone.
- ■ All routes: reject malformed IDs and unknown fields where appropriate; use centralized error handling.
- ■ All write routes: verify role/ownership and prevent mass assignment.
- ■ All list routes: cap page size and avoid unbounded queries.

A route inventory should list method, path, middleware, role, validation schema, and test evidence for every endpoint.

9. Security Hardening Checklist

- ■ `helmet()` configured with suitable production settings.
- ■ CORS allowlist restricted to known frontend origins.
- ■ Request body size limits configured.
- ■ Rate limiting on login, token refresh, and expensive analytics routes.
- ■ JWT secrets stored in environment variables; access tokens short-lived.
- ■ Passwords hashed with a suitable password hashing algorithm; never logged or returned.
- ■ Refresh tokens rotated/revoked correctly if refresh-token authentication is used.
- ■ Production errors do not expose stack traces or database internals.
- ■ Database account uses least-privilege permissions.
- ■ Dependencies audited and updated after reviewing breaking changes.
- ■ HTTPS enforced in production.
- ■ Logs exclude passwords, access tokens, refresh tokens, and sensitive personal data.

These are additional hardening checks; prioritize the assignment's named issues first.

10. Remediation and Retest Log

Complete this table with actual evidence.

Finding	Change made (file/route)	Regression test	Result	Status
F-01 Access control	___	AC-01 to AC-08	___	Open / Fixed
F-02 Mass assignment	___	IV-07 + role-change test	___	Open / Fixed
F-03 NoSQL injection	___	IV-01, IV-02	___	Open / Fixed

Finding	Change made (file/route)	Regression test	Result	Status
F-04 Validation gaps	___	IV-03 to IV-06, IV-08, IV-09	___	Open / Fixed
F-05 Error leakage	___	Force controlled server error	___	Open / Fixed
F-06 Analytics cost	___	Benchmark and cache HIT/MISS	___	Open / Fixed

A finding is **Fixed** only after the retest passes. Any unresolved critical/high issue must be listed as open with its impact and mitigation; do not claim full completion while one remains.

11. Final Client Summary Template

Use this summary only after you have completed the tests and filled the actual results:

Audit performed: [date, environment, commit/version]

Endpoints reviewed: [number]

Findings: Critical [#], High [#], Medium [#], Low [#]

Critical/high findings fixed and retested: [# of #]

Validation coverage: [# of # routes covered]

Analytics cache: [in-memory/Redis], TTL [seconds], measured hit rate [percentage]

Performance: Analytics p95 changed from [before] ms to [after] ms ([percentage] reduction); HTTP error rate changed from [before] to [after].

Remaining risks: [none, or list each item and mitigation]

Completion decision

The task meets its stated acceptance criteria only when:

1. The finding register is based on actual tests and severity is justified.
2. All critical/high findings are fixed and the same tests pass after remediation.
3. Every API endpoint has validation appropriate to its inputs.
4. Analytics caching is implemented with safe cache keys and a documented invalidation/staleness strategy.
5. Before/after performance numbers are measured under comparable conditions and include p95 latency.
6. The report contains reproducible evidence rather than assumed outcomes.

Appendix A — Useful Postman test setup

Create environment variables:

- `baseUrl`: API base URL, e.g. `http://localhost:5000`
- `adminToken`: valid admin access token
- `managerToken`: valid manager token
- `employeeToken`: valid employee token

For protected endpoints, add header:

```
Authorization: Bearer {{employeeToken}}
```

For each test, save the request and record:

• Request method/path and role used

• Status code

• Sanitized response body

• Whether any database state changed

• Pass/fail and retest date

Never put real production secrets or tokens in the report.

Appendix B — Example report references

• OWASP API Security Top 10: https://owasp.org/www-project-api-security/

• OWASP Input Validation Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

• MongoDB query optimization / explain: https://www.mongodb.com/docs/manual/reference/method/cursor.explain/

• Express production security best practices: https://expressjs.com/en/advanced/best-practice-security.html

• k6 documentation: https://grafana.com/docs/k6/latest/